МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Построение и анализ алгоритмов»

ТЕМА: Кратчайшие пути в графе: коммивояжёр.

Вариант: 7

Студент гр. 4343 _____ Гизатов Я.Р.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург

2026

Задание

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

- n – количество городов ($5 \leq n \leq 15$).
- Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j. Если $graph[i][j] = 0$ и $(i \neq j)$, это означает, что прямого пути между городами нет.

Выходные данные:

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.
- Если такого пути не существует, вывести "no path".

Sample Input 1:

5

0 1 13 23 7

12 0 15 18 28

21 29 0 33 28

23 19 34 0 38

5 40 7 39 0

Sample Output 1:

78

0 4 2 3 1 0

Sample Input 2:

3

0 1 0

1 0 1

0 1 0

Sample Output 2:

no path

Вариант 7

Точный метод: динамическое программирование (не МВиГ), рекурсивная реализация.

Приближённый алгоритм: АЛШ-1.

Требование перед сдачей: прохождение кода в задании 3.1 на Stepik. Замечание к варианту 7. АЛШ-1 начинать со стартовой вершины.

Выполнение работы

Программа реализует два подхода к решению задачи коммивояжёра: точный алгоритм (Хелда Карпа) на основе динамического программирования с битовыми масками и приближённый эвристический, алгоритм АЛШ-1. Оба алгоритма принимают на вход количество вершин (для точного алгоритма) и стартовую вершину (для приближенного алгоритма), матрицу смежностей, а выводят результат в соответствии с заданием.

Точный алгоритм (Хелда Карпа)

Основные структуры данных

vector<vector<int>> graph – двумерный вектор, представляющий матрицу смежности графа размером $n \times n$. Каждый элемент $graph[i][j]$ содержит вес ребра из вершины i в вершину j. Если ребро отсутствует, значение равно 0. Эта структура служит исходными данными для задачи коммивояжера.

vector<vector<int>> dp – двумерный вектор размером $(1 << n) \times n$, используемый для динамического программирования. Элемент $dp[mask][pos]$ хранит минимальную стоимость завершения маршрута из текущего состояния, где $mask$ это битовая маска посещенных вершин, а pos является текущей вершиной. Инициализируется значением -1 для невычисленных состояний.

vector<vector<int>> parent – двумерный вектор той же размерности $(1 << n) \times n$, предназначенный для восстановления оптимального пути. Элемент $parent[mask][pos]$ хранит номер следующей вершины, которую нужно посетить из состояния $(mask, pos)$, чтобы достичь минимальной стоимости маршрута. Значение -1 указывает на отсутствие следующей вершины.

Описание алгоритма

Первый код реализует решение задачи коммивояжера методом динамического программирования с использованием битовых масок

(алгоритм Хелда–Карпа). На вход алгоритм принимает из стандартного потока ввода целое число n – количество вершин графа, а затем матрицу смежности размером n × n, где каждый элемент $graph[i][j]$ задает вес ребра из вершины i в вершину j, а значение 0 обозначает отсутствие ребра. После считывания входных данных программа инициализирует основные структуры: двумерный вектор dp размером $(1 << n) \times n$, заполненный значениями -1 для мемоизации результатов подзадач, и двумерный вектор parent той же размерности, заполненный -1 для последующего восстановления оптимального маршрута. Затем запускается рекурсивная функция $tsp(mask, pos)$ с начальными параметрами $mask = 1$ (посещена только стартовая вершина 0) и $pos = 0$. Функция работает следующим образом: если маска mask равна $(1 << n) - 1$, то есть все вершины посещены, она возвращает вес ребра из текущей вершины pos обратно в стартовую вершину 0 (если ребро существует, иначе возвращает INF). Если значение для данной пары $(mask, pos)$ уже вычислено и сохранено в dp, оно возвращается из кэша. В противном случае функция перебирает все вершины next, которые еще не посещены (проверяется битовой маской) и для которых существует ребро из pos в $next$. Для каждого такого кандидата рекурсивно вычисляется стоимость завершения маршрута через вызов $tsp(new_mask, next)$ плюс вес ребра $graph[pos][next]$. Из всех вариантов выбирается тот, который дает минимальную суммарную стоимость, запоминается лучшая следующая вершина в $parent[mask][pos]$, а полученная минимальная стоимость сохраняется в $dp[mask][pos]$ и возвращается в вызывающий уровень рекурсии. После завершения работы рекурсивной функции в main получается значение result – минимальная стоимость полного цикла, начинающегося и заканчивающегося в вершине 0. Если результат не меньше INF, программа выводит сообщение no path. В противном случае выводится числовое значение стоимости, а затем вызывается функция $reconstruct_path()$, которая, используя сохраненный массив parent, восстанавливает оптимальный маршрут: начиная с маски

$mask = 1$ и позиции $pos = 0$, последовательно извлекает следующую вершину из $parent[mask][pos]$, добавляет ее в путь, обновляет маску и текущую позицию, и так до тех пор, пока маска не станет равной $(1 << n) - 1$. В конце в путь добавляется возврат в стартовую вершину 0. Итоговый маршрут выводится в стандартный поток вывода в виде последовательности вершин, а также записывается в лог-файл log.txt вместе с промежуточными состояниями рекурсии для отладки.

Оптимизации

1. **Использование битовых масок** – компактное представление подмножеств вершин в виде целых чисел позволяет эффективно хранить состояния и выполнять побитовые операции (проверка принадлежности, добавление вершины), что ускоряет работу по сравнению с использованием массивов или множеств.
2. **Мемоизация результатов** – таблица dp сохраняет уже вычисленные значения для каждой пары (mask, pos), что предотвращает повторные рекурсивные вызовы и сокращает временную сложность с факториальной до экспоненциальной.
3. **Хранение родительских вершин** – массив parent позволяет восстановить оптимальный маршрут без повторного перебора состояний, экономя время на постобработке.
4. **Раннее отсечение недостижимых переходов** – проверка $graph[pos][next] != 0$ исключает рассмотрение ребер, которые отсутствуют в графе, уменьшая количество перебираемых кандидатов.

Оценка сложности

Алгоритм использует таблицу мемоизации $dp[mask][pos]$, где mask – битовая маска посещённых вершин (всего $2n$ возможных масок), а pos – текущая вершина (n вариантов). Для каждого состояния (mask,pos) выполняется цикл по всем вершинам next от 0 до $n - 1$, чтобы найти непосещённую вершину и вычислить стоимость перехода. В

худшем случае перебираются все n вершин, и для каждой выполняется рекурсивный вызов (или обращение к уже вычисленному значению). Таким образом, общее количество операций пропорционально $2^n \cdot n \cdot n$. Рекурсия с мемоизацией гарантирует, что каждое состояние вычисляется ровно один раз.

- **Временная сложность:** $\boldsymbol{O(2^n \cdot n^2)}$

Пространственная сложность определяется хранением двух таблиц размером $2^n \cdot n$ dp и parent. Каждая таблица требует $O(2^n \cdot n)$ памяти. Дополнительная структура исходная матрица смежности занимает не более $O(n^2)$, что пренебрежимо мало по сравнению с экспоненциальным членом.

- **Пространственная сложность:** $\boldsymbol{O(2^n \cdot n)}$

Приближенный алгоритм (АЛШ-1)

vector<vector<double>> matr; – двумерный вектор, представляющий матрицу весов графа. Размерность определяется количеством строк, прочитанных из входных данных. Каждый элемент $matr[i][j]$ содержит вес ребра из вершины i в вершину j. Используется для хранения исходных данных задачи коммивояжера и быстрого доступа к весам дуг при вычислении оценок и стоимости маршрута.

vector<bool> visited; – одномерный вектор логических значений размером N , где N это количество вершин графа. Каждый элемент $visited[i]$ принимает значение true, если вершина i уже включена в строящийся маршрут, и false в противном случае. Используется для отслеживания множества уже посещенных вершин на каждом шаге алгоритма.

vector<int> path; – одномерный вектор целых чисел, хранящий последовательность вершин в порядке их добавления в маршрут. Начинается со стартовой вершины и постепенно пополняется новыми вершинами в процессе работы алгоритма. По завершении содержит все вершины графа в порядке обхода (без замыкающей дуги).

Описание алгоритма

Данный код реализует упрощенную версию приближенного алгоритма построения маршрута коммивояжера, использующего жадную стратегию с нижней оценкой, но с сокращенной функцией вычисления оценки. На вход алгоритм принимает из стандартного потока ввода целое число start_ver – номер стартовой вершины, а затем матрицу весов графа, которая читается построчно до конца входных данных. Каждая строка разбивается на числа с помощью istringstream, и значения добавляются в матрицу matr, после чего количество вершин N определяется как размер прочитанной матрицы, а стартовый индекс start_idx устанавливается равным введенному значению. Далее выполняется инициализация структур: вектор visited размером N заполняется значениями false для отслеживания посещенных вершин, вектор path начинает формироваться с добавления стартовой вершины, которая также отмечается как посещенная, переменная current_cost инициализируется нулем для накопления стоимости маршрута, переменная last устанавливается на стартовую вершину, и открывается файл logALSH.txt для записи подробного лога выполнения алгоритма.

Основной цикл выполняется, пока количество вершин в пути меньше общего числа вершин N. На каждом шаге сначала проверяется, осталась ли только одна непосещенная вершина: если $path.size() == N - 1$, то эта последняя вершина находится простым перебором, вычисляется вес ребра из текущей концевой вершины last в эту последнюю вершину, вершина добавляется в путь, отмечается как посещенная, current_cost увеличивается на вес ребра, last обновляется, и цикл прерывается. Если же осталось более одной непосещенной вершины, алгоритм выполняет выбор следующей вершины на основе минимизации оценочной функции. Для каждого кандидата j, который еще не посещен, вычисляется вес ребра $s = matr[last][j]$ из текущей концевой вершины в кандидата, создаются временные копии векторов visited и path с добавлением кандидата,

вызывается функция compute_L(new_visited, new_path, matr), которая вычисляет нижнюю оценку стоимости завершения маршрута, после чего оценочное значение $value = s + L$ сравнивается с текущим лучшим. В упрощенной версии функции compute_L выполняется поиск минимального входящего ребра для стартовой вершины пути (исключая саму стартовую и конечную) и минимального исходящего ребра для конечной вершины пути (исключая конечную и стартовую), эти значения суммируются и делятся на два, при этом отсутствует учет двух наименьших исходящих ребер для непосещенных вершин, что делает оценку менее точной, но значительно ускоряет вычисления.

Из всех кандидатов выбирается тот, для которого value минимально. Для выбранной вершины best_next фиксируется вес ребра s, вершина добавляется в путь, отмечается как посещенная, current_cost увеличивается, last обновляется. На каждом шаге в лог-файл записывается текущее состояние пути, рассматриваемые кандидаты, их веса и оценки, а также принятое решение. После завершения основного цикла, когда все вершины собраны в путь, алгоритм замыкает маршрут: к текущей суммарной стоимости добавляется вес ребра из последней вершины last обратно в стартовую вершину start_idx. В лог-файл записывается информация о замыкающей дуге, полный маршрут (последовательность вершин с возвратом к стартовой) и итоговая стоимость. Выходными данными программы являются последовательность вершин в порядке обхода (без дублирования стартовой в конце) и итоговая стоимость построенного маршрута: первая строка вывода содержит номера вершин через пробел, вторая строка – числовое значение общей стоимости.

Оценка сложности

Чтение входной матрицы занимает $O(n^2)$. Основной цикл выполняется n раз (добавление каждой вершины в маршрут). На каждом шаге, кроме последнего, для каждого из k непосещенных кандидатов (в худшем случае $k = O(n)$ выполняются:

- создание копий векторов new_visited и new_path размером $O(n)$ каждая;
- вызов функции compute_L, которая за один проход по всем N вершинам находит два минимума (входящее ребро для стартовой вершины и исходящее для конечной), работая за $O(n)$.

Таким образом, обработка одного кандидата требует $O(n)$ операций, а на одном шаге перебирается до N кандидатов, что даёт $O(n^2)$ на шаг. Суммируя по N шагам, получаем $O(n^3)$. Последний шаг (добавление последней вершины) выполняется за $O(n)$ и не влияет на асимптотику.

- **Временная сложность: $\boldsymbol{O(n^3)}$**

Основной объем памяти занимает хранение матрицы весов matr размером $n \times n$ – $O(n^2)$. Дополнительные структуры: векторы visited и path требуют $O(n)$ памяти каждая. Временные копии new_visited и new_path создаются в цикле, но их размер также ограничен $O(N)$, и они не накапливаются одновременно. Таким образом, общая потребляемая память определяется матрицей и составляет $O(n^2)$.

- **Пространственная сложность: $\boldsymbol{O(n^2)}$**

Тестирование

Точный алгоритм (Хелда Карпа)

Тест 1 – Проверка метода на матрице размера 5

Ввод:

```
5
0 1 13 23 7
12 0 15 18 28
21 29 0 33 28
23 19 34 0 38
5 40 7 39 0
```

Вывод:

```
5
0 2 1 0
```

Тест 2 – Проверка для полного случайного графа размера 7

Ввод:

```
7
76 70 5 18 38 18 67
12 80 40 43 43 79 17
8 52 43 60 27 41 18
48 70 30 10 63 33 55
82 26 35 84 73 33 97
68 6 52 89 22 17 84
26 84 70 24 45 86 7
```

Вывод:

```
140
0 2 6 3 5 4 1 0
```

Тест 3 – Проверка метода на матрице без пути

Ввод:

```
5
0 0 0 0 0
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

Вывод:

```
no path
```

Тест 4 – Проверка метода на минимальной матрице

Ввод:

```
1
1
```

Вывод:

```
1
0 0
```

Тест 5 – Проверка метода на максимальной матрице

Ввод:

```
15
17 84 41 87 58 99 20 63 28 67 97 55 69 27 54
7 16 16 84 14 53 73 26 74 64 44 93 4 78 98
9 14 92 85 78 74 84 3 75 66 72 32 95 87 2
80 20 35 4 73 44 26 16 67 8 62 58 58 18 50
16 60 84 19 58 28 45 86 35 0 41 57 35 57 59
34 69 91 6 76 42 49 10 87 18 22 20 3 72 67
89 20 28 66 32 16 60 49 72 63 44 28 49 80 23
10 12 44 38 64 8 56 44 90 53 68 51 2 28 8
13 86 74 41 92 61 85 99 54 65 22 94 78 38 69
23 51 7 59 66 8 25 54 15 28 9 22 37 85 56
16 51 85 55 57 100 71 8 70 53 23 85 46 97 24
51 83 66 42 93 37 76 96 74 73 69 53 13 40 36
52 33 55 7 78 75 31 91 22 52 54 63 54 24 881
65 99 16 88 21 31 87 29 42 67 97 77 99 10 73
30 60 44 62 64 0 70 66 6 66 49 89 76 56 85
```

Вывод:

```
200
0 6 1 4 3 9 10 7 5 11 12 13 2 14 8 0
```

Приближенный алгоритм (АЛШ-1)

Тест 1 – Проверка метода на максимальной матрице

Ввод:

```
15
62 97 94 76 79 92 11 45 44 52 79 43 31 7 73
70 55 96 3 94 19 90 100 47 47 15 7 45 1 84
89 17 56 81 38 99 89 76 42 26 96 95 89 34 78
49 47 1 70 2 93 88 92 11 90 76 64 14 55 55
50 15 70 65 12 3 84 40 35 71 88 80 62 25 68
54 36 13 60 3 27 10 46 36 64 10 28 8 35 26
7 69 70 50 1 83 64 32 44 99 62 67 41 34 12
54 5 86 55 83 48 97 21 39 78 55 51 32 78 83
94 61 11 89 36 33 47 61 50 50 70 98 8 94 86
32 97 78 4 81 36 23 95 60 94 6 42 61 50 38
8 42 35 80 63 60 26 28 99 72 93 52 19 87 82
66 14 54 63 13 18 48 35 50 68 97 90 5 91 10
62 89 61 69 78 88 53 68 49 98 97 82 37 57 57
73 2 90 91 21 46 43 76 72 99 11 33 82 64 65
14 70 4 32 35 65 97 56 19 61 4 55 60 70 52
```

Вывод:

```
0 13 1 3 4 5 6 14 2 9 10 7 8 12 11 0
298
```

Тест 2 – Проверка метода на простой матрице

Ввод:

```
3
0 0 1
2 3 0
5 2 0
```

Вывод:

```
0 2 1 0
5
```

Тест 3 – Проверка метода на матрице без пути

Ввод:

```
5
0 0 0 0 0
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

Вывод:

```
no path
```

Тест 4 – Проверка метода на минимальной матрице

Ввод:

```
1
1
```

Вывод:

```
0 0
1
```

Тест 2 – Проверка метода на произвольной матрице

Ввод:

```
8
60 70 10 45 28 3 55 70
67 44 10 6 84 3 0 6
99 84 53 11 88 58 16 45
73 49 93 26 19 0 45 94
68 95 40 31 2 68 54 37
24 62 77 56 76 94 63 72
64 82 39 24 37 53 18 65
77 83 21 31 92 74 83 79
```

Вывод:

```
0 2 3 4 7 1 5 6 0
290
```

Программы для всех случаев отработала корректно.

Вывод

В ходе выполнения лабораторной работы были реализованы два алгоритма решения задачи коммивояжёра: точный метод на основе динамического программирования с битовыми масками (алгоритм Хелда–Карпа) и приближённый эвристический алгоритм АЛШ-1. Точный алгоритм обеспечивает нахождение оптимального гамильтонова цикла за время $O(2^{\mathrm{n}} \cdot n^2)$ и гарантирует корректный результат для графов размером до 15 вершин, однако его применение ограничено экспоненциальным ростом времени вычислений. Приближённый алгоритм, работающий за $O(n^3)$, позволяет строить допустимые маршруты для графов большего размера за приемлемое время, но не гарантирует оптимальности решения. Проведённое тестирование на различных наборах входных данных подтвердило корректность работы обеих реализаций: точный алгоритм успешно находит минимальный маршрут либо сообщает о его отсутствии, а приближённый строит путь, который в большинстве случаев оказывается приемлемым по стоимости. Таким образом, поставленная задача полностью выполнена, программы соответствуют требованиям варианта и могут применяться для решения задачи коммивояжёра в зависимости от требуемой точности и размерности входных данных.

Приложение

Код программы:

ALSH-1.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>
#include <limits>
#include <fstream>

using namespace std;

const double INF = numeric_limits<double>::infinity();

double compute_L(const vector<bool>& visited, const vector<int>& path,
const vector<vector<double>>& matr) {
    int N = matr.size();
    double total = 0.0;

    int start = path.front();
    int end = path.back();

    double min_in = INF;
    double min_out = INF;
    for (int u = 0; u < N; ++u) {
        if (u != start && u != end) {
            double w = matr[u][start];
            if (w < min_in && w != 0) min_in = w;
            w = matr[end][u];
            if (w < min_out && w != 0) min_out = w;
        }
    }

    total += min_in + min_out;

    return total / 2.0;
}

int main() {
    int start_ver;
    cin >> start_ver;
    cin.ignore();
    start_ver = 0;

    vector<vector<double>> matr;
    string line;
    while (getline(cin, line)) {
        if (line.empty()) continue;
        vector<double> row;
        istringstream iss(line);
        double val;
        while (iss >> val) {
            row.push_back(val);
        }
        matr.push_back(row);
    }

    int N = matr.size();
    if (N == 0) {
```

```
            cout << "no path" << endl;
            return 0;
        }

        int start_idx = start_ver;

        vector<bool> visited(N, false);
        vector<int> path;
        path.push_back(start_idx);
        visited[start_idx] = true;

        double current_cost = 0.0;
        int last = start_idx;

        ofstream logFile("logALSH.txt");

        logFile << "\nНачало алгоритма\n";
        logFile << "Стартовая вершина: " << start_ver << "\n\n";

        while (path.size() < N) {
            logFile << "Шаг " << path.size() << endl;
            logFile << "Текущий путь: ";
            for (int v : path) logFile << v << " ";
            logFile << "\nТекущая концевая вершина: " << last << "\n";

            if (path.size() == N - 1) {
                int next = -1;
                for (int i = 0; i < N; ++i) {
                    if (!visited[i]) {
                        next = i;
                        break;
                    }
                }
                if (next == -1) {
                    logFile << "Ошибка: не найдена непосещённая вершина"
<< endl;
                    cout << "no path" << endl;
                    return 0;
                }
                double s = matr[last][next];
                if (s == 0) {
                    logFile << "Нет ребра из " << last << " в " << next <<
endl;
                    cout << "no path" << endl;
                    return 0;
                }
                logFile << "Последняя вершина: " << next << ", вес: " << s
<< "\n";
                path.push_back(next);
                visited[next] = true;
                current_cost += s;
                last = next;
                break;
            }

            double best_value = INF;
            int best_next = -1;

            logFile << "Рассматриваем кандидатов:\n";
            for (int j = 0; j < N; ++j) {
                if (!visited[j]) {
                    double s = matr[last][j];
                    if(s == 0) continue;
                    vector<bool> new_visited = visited;
```

```
                    new_visited[j] = true;

                    vector<int> new_path = path;
                    new_path.push_back(j);
                    double L = compute_L(new_visited, new_path, matr);
                    double value = s + L;

                    logFile << "Вершина " << j << ": s=" << s << ", L=" <<
L << ", s+L=" << value;
                    if (value < best_value) logFile << " (лучший пока)";
                    logFile << "\n";

                    if (value < best_value) {
                        best_value = value;
                        best_next = j;
                    }
                }
            }

            if (best_next == -1) {
                logFile << "Нет доступных вершин для продолжения пути из "
<< last << endl;
                cout << "no path" << endl;
                return 0;
            }

            double s = matr[last][best_next];
            logFile << "\nВыбрана вершина " << best_next << " (s=" << s <<
", s+L=" << best_value << ")\n";
            path.push_back(best_next);
            visited[best_next] = true;
            current_cost += s;
            last = best_next;
            logFile << "Текущая стоимость пути: " << current_cost << "\n\n";
        }

        double closing_cost = matr[last][start_idx];
        if (closing_cost == 0) {
            logFile << "Нет замыкающего ребра из " << last << " в " <<
start_idx << endl;
            cout << "no path" << endl;
            return 0;
        }
        current_cost += closing_cost;
        logFile << "Замыкающая дуга: " << last << " -> " << start_ver <<
", вес: " << closing_cost << "\n";
        logFile << "Полный путь: ";
        for (int v : path) logFile << v << " ";
        logFile << start_ver << "\n";
        logFile << "Общая стоимость: " << current_cost << endl;

        logFile.close();

        for (int v : path) {
            cout << v << " ";
        }
        cout << start_ver << endl;
        cout << current_cost << endl;

        return 0;
    }
```

HELD_CARP.cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
#include <fstream>
#include <bitset>

using namespace std;

const int INF = INT_MAX / 2;

int n;
vector<vector<int>> graph;
vector<vector<int>> dp;
vector<vector<int>> parent;

ofstream logFile;

string toBinary(int num, int bits) {
    string result(bits, '0');
    for (int i = 0; i < bits; i++) {
        if (num & (1 << (bits - 1 - i))) {
            result[i] = '1';
        }
    }
    return result;
}

int tsp(int mask, int pos) {

    logFile << "tsp(" << mask << "," << pos << ")  " << "mask: " <<
toBinary(mask, n) << endl;

    if (mask == (1 << n) - 1) {
        int ret = (graph[pos][0] != 0) ? graph[pos][0] : INF;
        logFile << "  base = " << ret << "\n";
        return ret;
    }

    if (dp[mask][pos] != -1) {
        logFile << "  memo = " << dp[mask][pos] << "\n";
        return dp[mask][pos];
    }

    int min_cost = INF;
    int best_next = -1;

    for (int next = 0; next < n; next++) {
        if (!(mask & (1 << next)) && graph[pos][next] != 0) {
            int new_mask = mask | (1 << next);
            int cost = graph[pos][next] + tsp(new_mask, next);

            if (cost < min_cost) {
                min_cost = cost;
                best_next = next;
                logFile << "   best " << next << " cost = " << min_cost
<< "\n";
            }
        }
    }

    dp[mask][pos] = min_cost;
    if (best_next != -1) parent[mask][pos] = best_next;
```

```
        logFile << "  dp[" << mask << "][" << pos << "] = " << min_cost <<
" parent = " << best_next << " mask: " << toBinary(mask, n) << "\n";

        return min_cost;
    }


    vector<int> reconstruct_path() {
        const int k = n;
        logFile << "Build result:" << endl;
        vector<int> path;
        int mask = 1;
        int pos = 0;
        path.push_back(0);
        logFile << "vertex: " << pos << " mask: " << toBinary(mask, n) <<
endl;
        while (mask != (1 << n) - 1) {
            int next = parent[mask][pos];

            if (next == -1) break;
            path.push_back(next);
            logFile << "next: " << next << endl;
            mask |= (1 << next);
            pos = next;
        }
        path.push_back(0);
        return path;
    }

    int main() {

        logFile.open("log.txt");

        cin >> n;
        graph.resize(n, vector<int>(n));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                cin >> graph[i][j];

        dp.resize(1 << n, vector<int>(n, -1));
        parent.resize(1 << n, vector<int>(n, -1));

        int result = tsp(1, 0);
        logFile << "Result: " << (result >= INF ? "no path" :
to_string(result)) << "\n";

        if (result >= INF) {
            cout << "no path" << endl;
        } else {
            cout << result << endl;
            vector<int> path = reconstruct_path();
            for (size_t i = 0; i < path.size(); i++) {
                cout << path[i];
                if (i != path.size() - 1) cout << " ";
            }
            cout << endl;
            logFile << "Path: ";
            for (size_t i = 0; i < path.size(); i++) logFile << path[i] <<
" ";
            logFile << "\n";
        }

        logFile.close();
        return 0;
```

```
}
```

gen_matr.py

```
import random
import os

def main():
    n = int(input("Введите размер квадратной матрицы: "))

    matrix = [[random.randint(0, 100) for _ in range(n)] for _ in
range(n)]

    os.makedirs("tests", exist_ok=True)

    with open("tests/testr", "w", encoding="utf-8") as f:
        f.write(f"{n}\n")
        for row in matrix:
            f.write(" ".join(map(str, row)) + "\n")

if __name__ == "__main__":
    main()
```